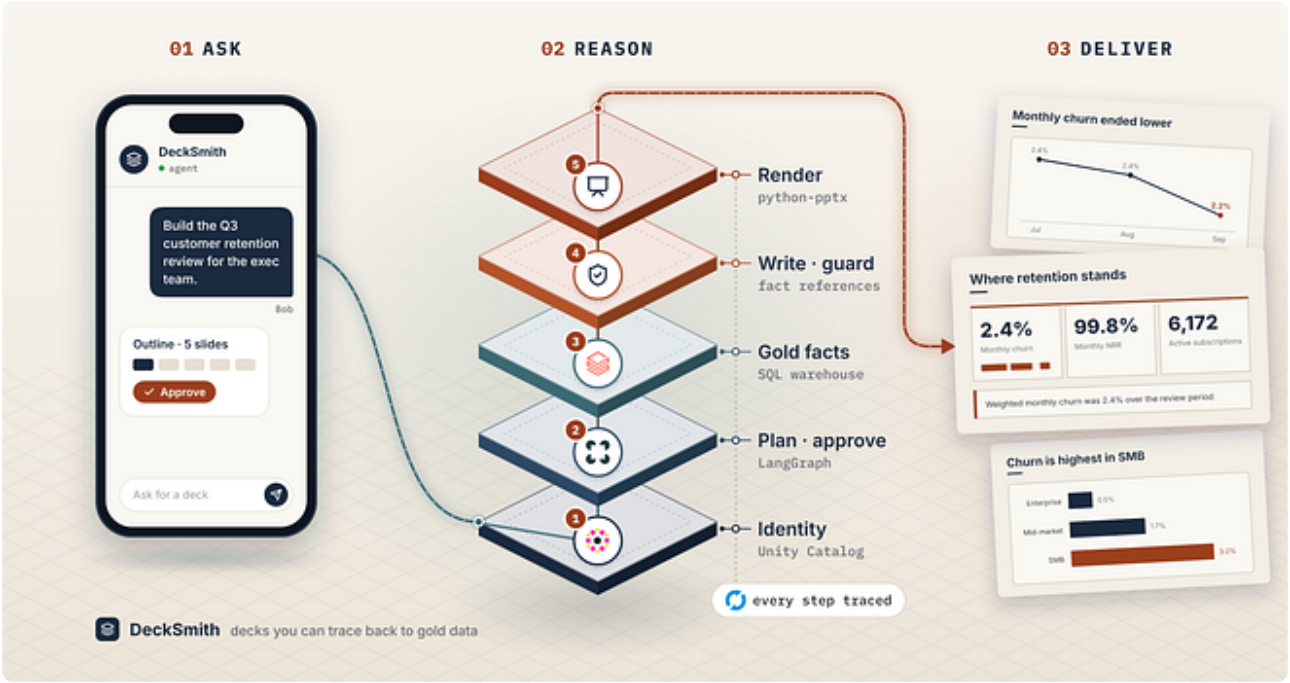


Build Decks Your Business Can Trust from Gold Data Products on Databricks

How I turned governed gold tables into editable slides, with every number traced to its query and every query run as the user who asked.

Santosh Shinde

Published October 18, 2026

Free: No

Read on Freedium · original

Contents

Introduction 0

The problem: trusted decks from gold data 0

Requirements, and how I proved each one 0

Kasal or DeckSmith? 0

Start local, shaped like Databricks 0

System design: one agent, two deployments 0

Step by step: from request to deck 0

1. Approved SQL templates 0

2. An outline, and a human in the loop 0

3. Facts, fetched as the user 0

4. The writer references typed facts 0

5. A deterministic renderer 0

6. One gateway for every model call 0

Defining the review metrics 0

Guardrails: trust by construction 0

Testing it end to end 0

The Databricks deployment design 0

Agent hosting 0

What still needs a workspace test 0

Lessons for a team starting this 0

What's next 0

Try it yourself 0

Final thought 0

Introduction

Recently I was working on a use case, and someone asked me a simple question: "We have our data in a data mesh on Databricks. How do I present it to my business?"

The people he meant don't open notebooks or dashboards. They open slides. So the real question was: can we turn gold data products into slides business users can trust?

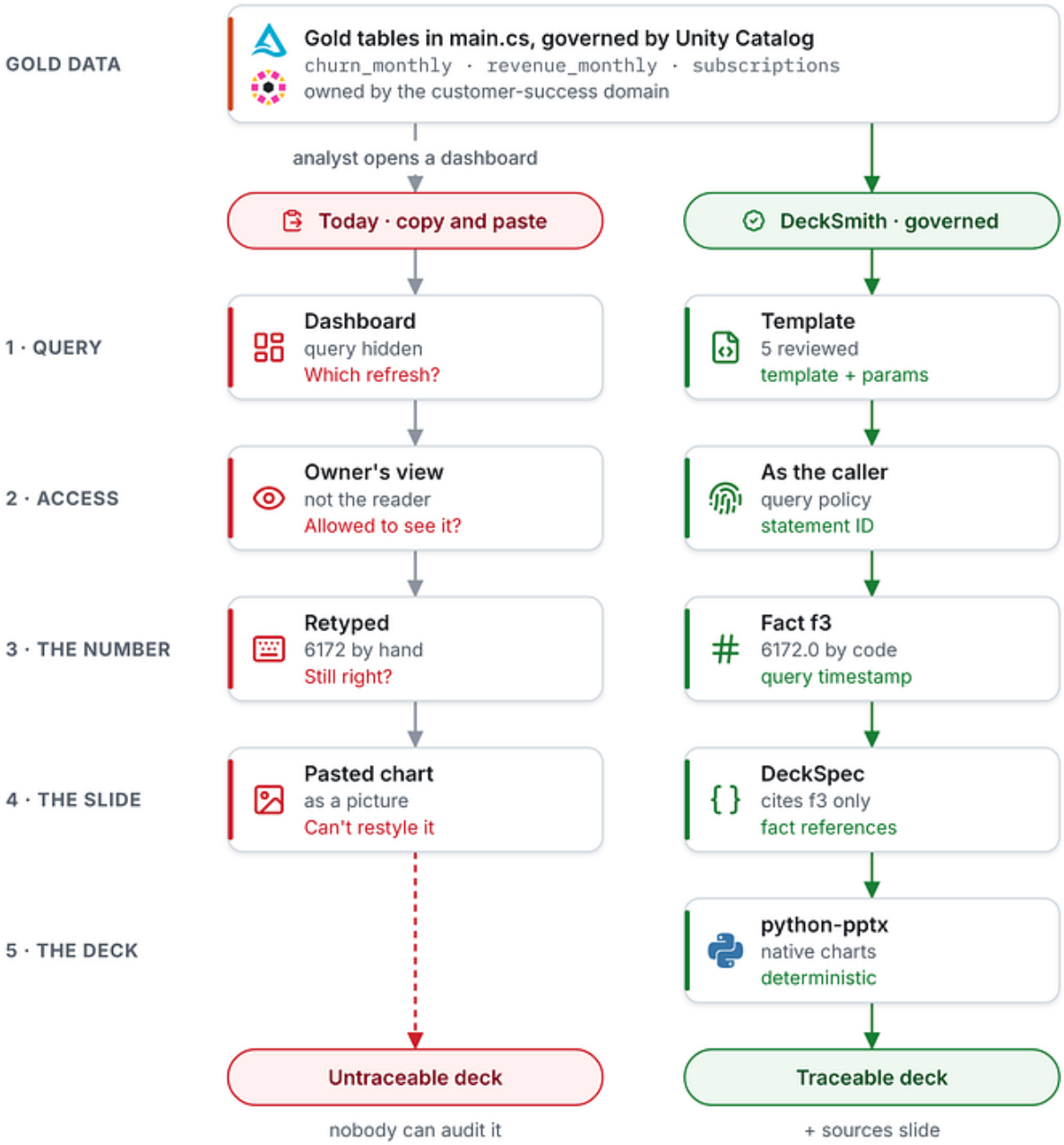
I wanted to start locally, on a lakehouse that behaves like Databricks, and treat it as system design for a real use case.

This build log follows Build a Document Knowledge Base on Databricks (https://freedium-mirror.cfd/https://medium.com/data-engineer-things/build-a-document-knowledge-base-on-databricks-2e2914fb8d4e). That piece was about who may retrieve which page. This one is about who may see which row. The rule is the same: the caller's permissions travel with every query.

A note before I start: the repo, deck-smith-local (https://github.com/santoshshinde2012/deck-smith-local), is open source and its data is synthetic (seed 42, 8,000 subscriptions). The local stack is built and tested. The Databricks deployment is my design, not something I have run in production. I wrote this with AI writing assistance and checked every number, path and command against the repo.

The problem: trusted decks from gold data

In a data mesh, each domain ends in "insights ready for consumption by business users" (Databricks blog (https://www.databricks.com/blog/building-data-mesh-based-databricks-lakehouse-part-2)). That last mile usually ends up in a slide, where governance stops. An analyst copies a churn rate from a dashboard, pastes a chart as an image, and hopes it was the latest refresh. Nobody can tell which query produced a figure, or whether the author was allowed to see those rows.



Same source on both paths; everything after it is different.
Query time is recorded. Downloaded decks are static; recipient access and source freshness are not rechecked.

Generative AI makes this faster and riskier. In one study of LLM summaries of 10-K reports, about 5% of the numbers were wrong (Cao et al. (https://arxiv.org/abs/2404.06162)). On financial tables, the strongest models evaluated in the study got 10–20% of its multi-step calculations wrong (FAITH (https://arxiv.org/abs/2508.05201)). One wrong number in an exec deck, and the room stops believing the rest.

So my problem statement had four parts:

1. Business users need decks built only from gold data products.
2. Every number must be traceable to the query that produced it.

3. A generated deck contains only data visible to the requesting user when its queries run.
4. Prove the workflow locally, then validate the deployment separately on Databricks.

The use case is the quarterly churn review for the exec team. Three synthetic tables stand in for the gold data products: `churn_monthly`, `revenue_monthly` and `subscriptions`.

Requirements, and how I proved each one

I turned the trust requirements into checks, then used review failures to tighten them.

REQUIREMENT	HOW DECKSMITH SOLVES IT	PROOF IN THE REPO
No LLM-written SQL	The planner picks one of five reviewed SQL templates	Unknown templates and bad parameters fail before approval, so no query runs
Numeric metrics come from query results	Scalar placeholders and chart series reference typed facts; QA checks every writer text field for bare digits	Regression tests reject invented legend percentages, series used as scalars and mismatched chart categories
Each user sees only their rows	Every query runs as the signed-in user	bob sees 6,172 active subscriptions, alice 2,184, carol no deck
A person approves the plan	The agent pauses before queries run and checks the final slide IDs, order and layouts against the approved outline	The edited run resumes and renders four slides; writer changes to the structure are refused
A deck the business can use	Native charts, a required sources appendix and query details in notes	The default deck has two charts; the edited example has one, with every plotted value checked
API runs stay private	An owner check on every run or file read	Other users' runs are hidden; anonymous calls are rejected
Local first, a path to Databricks	Swappable components, wired in one place	Architecture and mocked adapter tests; workspace deployment, networking and managed services remain unverified

Kasal or DeckSmith?

Kasal (https://www.databricks.com/blog/introducing-kasal) is a Databricks Labs project: a low-code builder for CrewAI agent workflows that you can install as a Databricks App. It already has on-behalf-of-user authentication and MLflow tracing, and its repo (https://github.com/databrickslabs/kasal) includes a PowerPoint export. What I needed was narrower and stricter. In DeckSmith, numeric metrics reference typed facts that trace back to approved queries. Code checks those references before

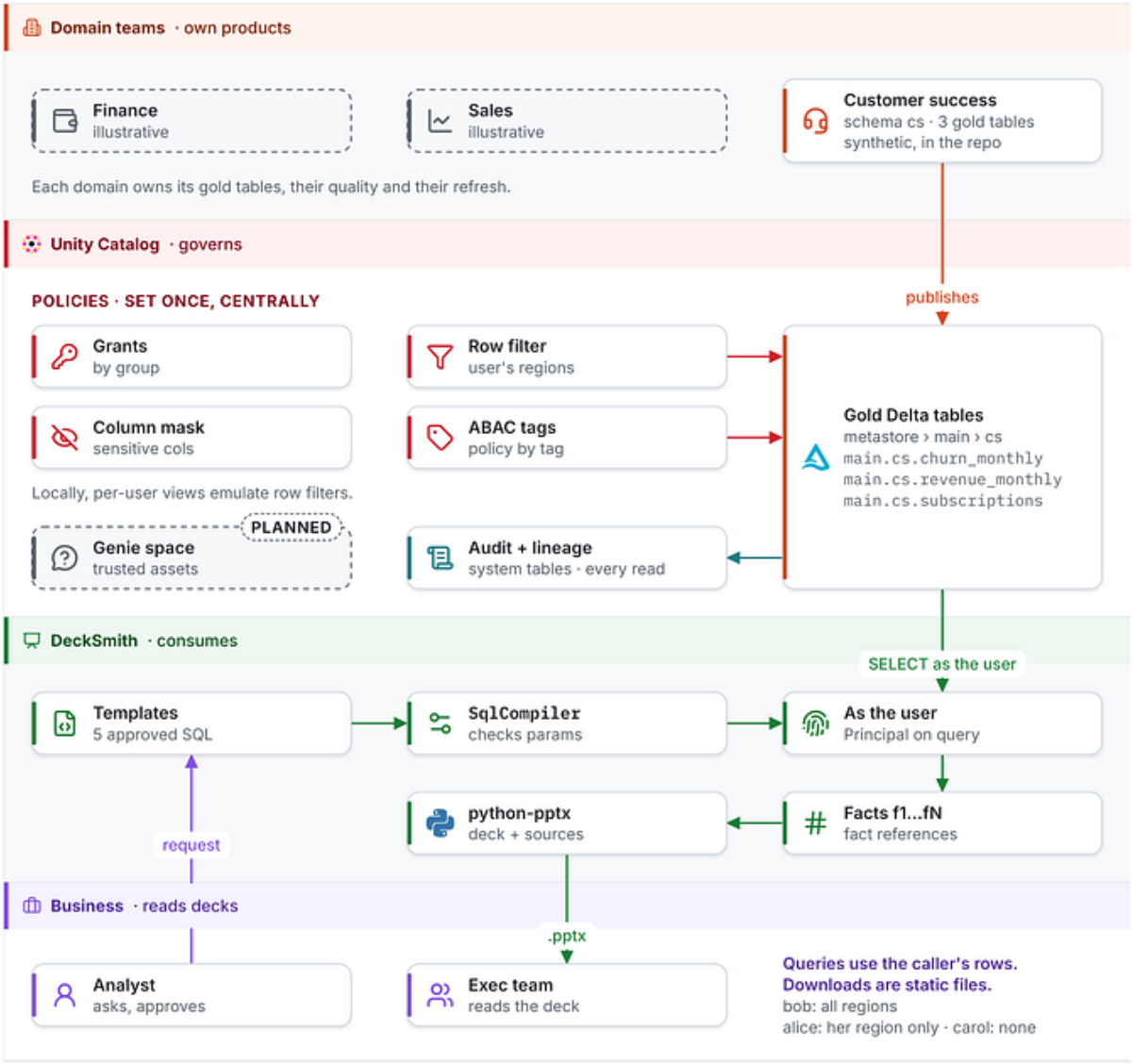
rendering. The run also pauses for a person's approval before any business-data query runs. If you want non-coders composing agent workflows on a canvas, start with Kasal. For one governed deck pipeline, owning the code was the smaller system.

Start local, shaped like Databricks

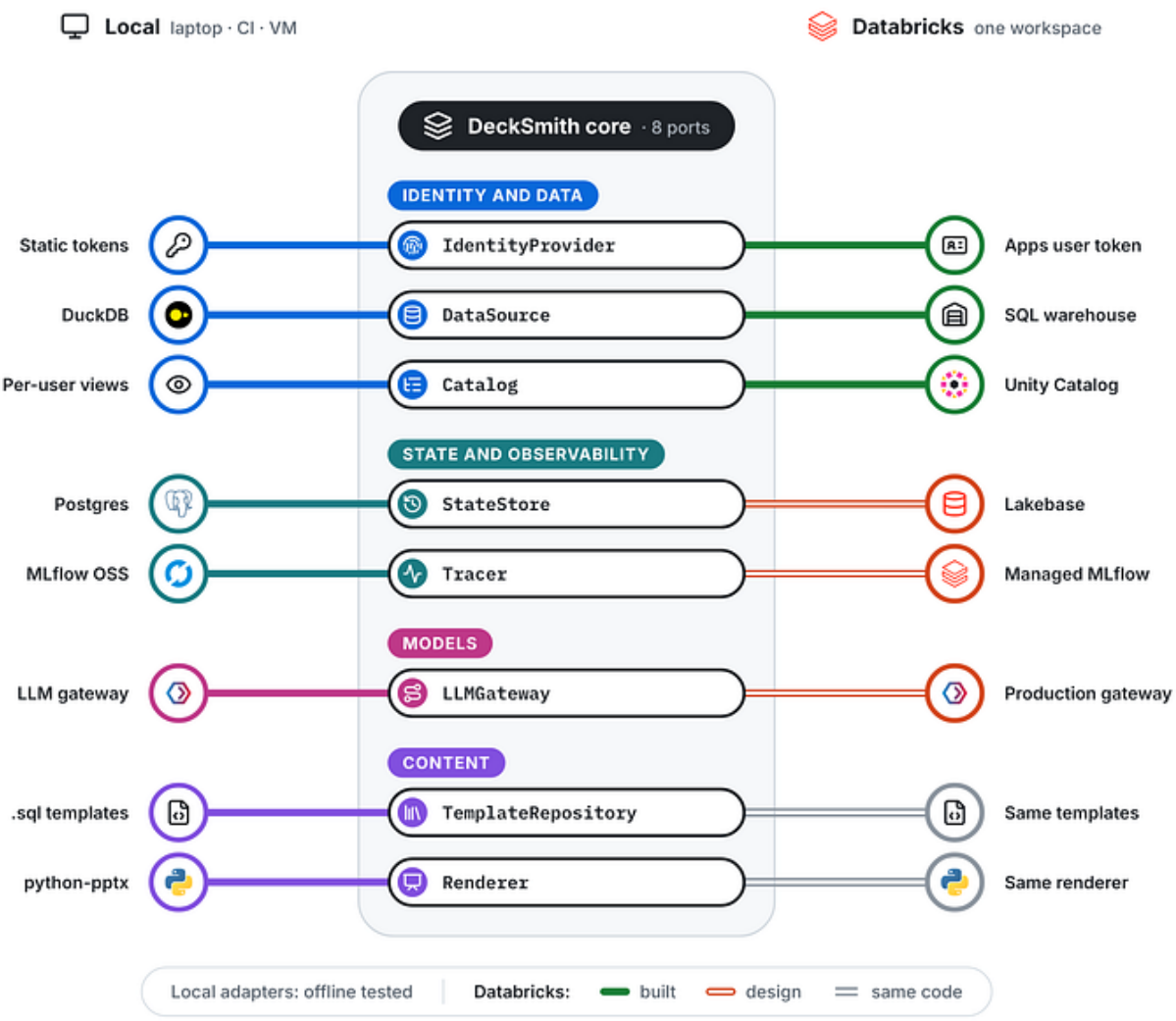
I didn't want "works on my laptop" to mean "only works on my laptop". Locally, DuckDB, Postgres, MLflow and an open-source LLM gateway stand in for the SQL warehouse, Lakebase, managed MLflow and the production gateway. The SQL templates are written once, in Databricks SQL, and sqlglot (https://sqlglot.com/sqlglot.html) translates them for DuckDB. A view per user stands in for Unity Catalog row filters. The views match for my three test users, but they are an emulation, not a security boundary.

System design: one agent, two deployments

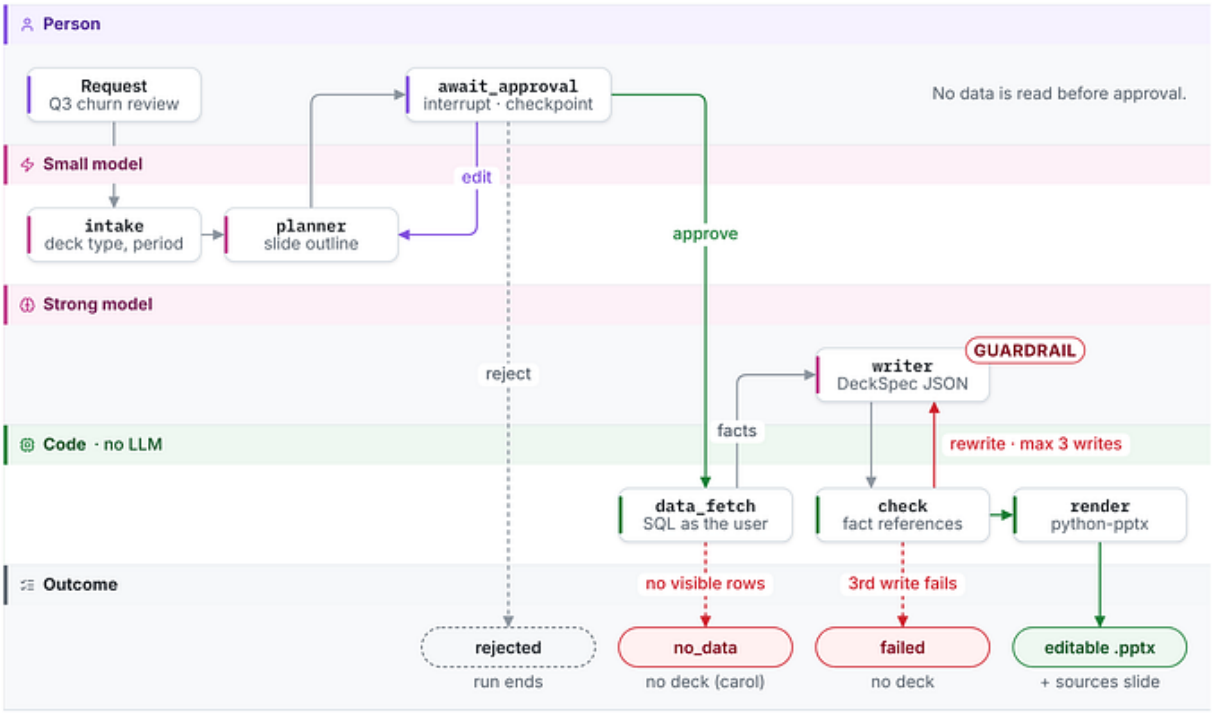
DeckSmith consumes data products and owns none of them. Domain teams own the gold tables, Unity Catalog holds the policies, and DeckSmith reads only through approved templates, as the person who asked.



The core of the app reaches the outside world only through eight small interfaces, called ports, such as the data source, the LLM gateway and identity. Each port has a local adapter and, where it matters, a Databricks one. One module, `bootstrap.py`, wires them together. Settings select the adapters; a workspace deployment still needs app resources, identity scopes, Lakebase credentials, a tracing destination, artifact storage and network configuration. Even the data source port takes the user as an argument: no query runs without saying who it is for.



The agent is a LangGraph (https://docs.langchain.com/oss/python/langgraph/overview) state machine, and the important part is where the LLM is not used. A small model plans. A person approves. Code fetches the data. A strong model writes. Code checks and renders.



Step by step: from request to deck

The runs in this article use the repo's deterministic stand-in model, so they show the pipeline and its checks, not a real model's writing.

1. Approved SQL templates

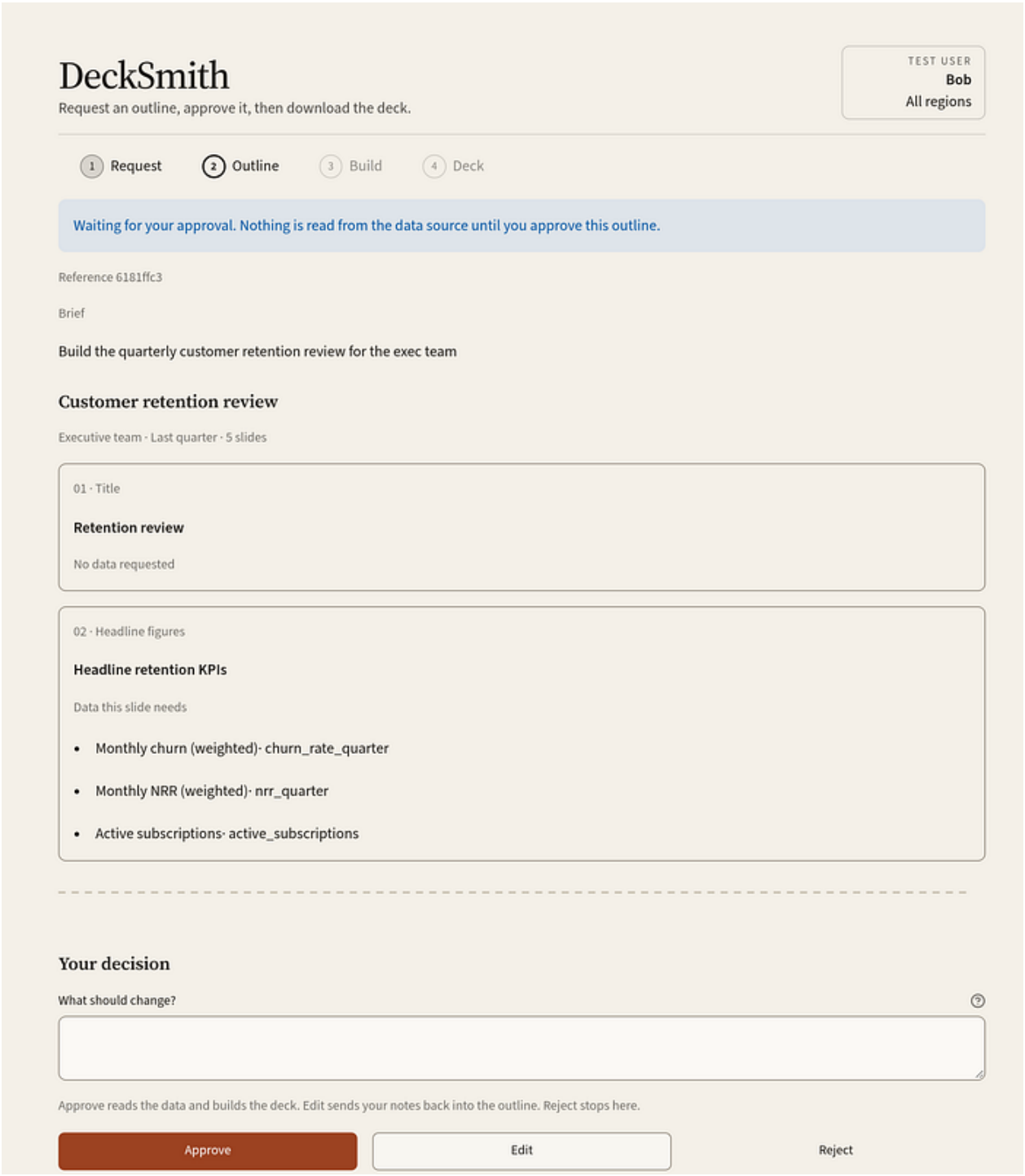
Each question the business can ask is a reviewed template with typed parameters. The repo ships five. Here is churn by segment:

```
-- src/decksmith/sql/templates/churn_by_segment.sql
-- id: churn_by_segment
-- description: Weighted monthly churn by customer segment over a period; not cumulative
cohort churn
-- kind: series
-- format: pct1
-- unit: ratio
-- param: start_month date [0-9]{4}-[0-9]{2}-01
-- param: end_month date [0-9]{4}-[0-9]{2}-01
SELECT segment,
       try_divide(SUM(churned), SUM(active_start)) AS churn_rate
FROM {{table:cs.churn_monthly}}
WHERE `month` >= :start_month AND `month` < :end_month
GROUP BY segment
ORDER BY segment
```

The planner only picks a template and fills in its parameters. An unknown template or a bad parameter rejects the whole plan before approval, so no query runs. A new question means a new template, reviewed like code.

2. An outline, and a human in the loop

Bob asks: "Build the quarterly customer retention review for the exec team." The agent proposes five slides (title, headline KPIs, monthly churn trend, monthly churn by segment, sources) and stops.



The Streamlit UI shows who is signed in (Bob, all regions) and a four-step bar: Request, Outline, Build, Deck. The outline is headed "Executive team · Last quarter · 5 slides". While the run waits, it says: "Waiting for your approval. Nothing is read from the data source until you approve this outline." Each proposed slide is a card that lists the data it needs.

The paused run is saved, so it can wait for an answer. Bob can approve, reject, or edit ("drop the segment slide"), which sends it back to the planner. In the UI, Approve reads the data and builds the deck, Edit sends his notes back into the outline, and Reject stops the run. No business-data query has run yet. After Bob drops the segment slide, approving that same run produces four slides and one chart. The default unedited deck has five slides and two charts.

3. Facts, fetched as the user

After approval, each template runs as the user. Locally, each user reads through their own view of the data.

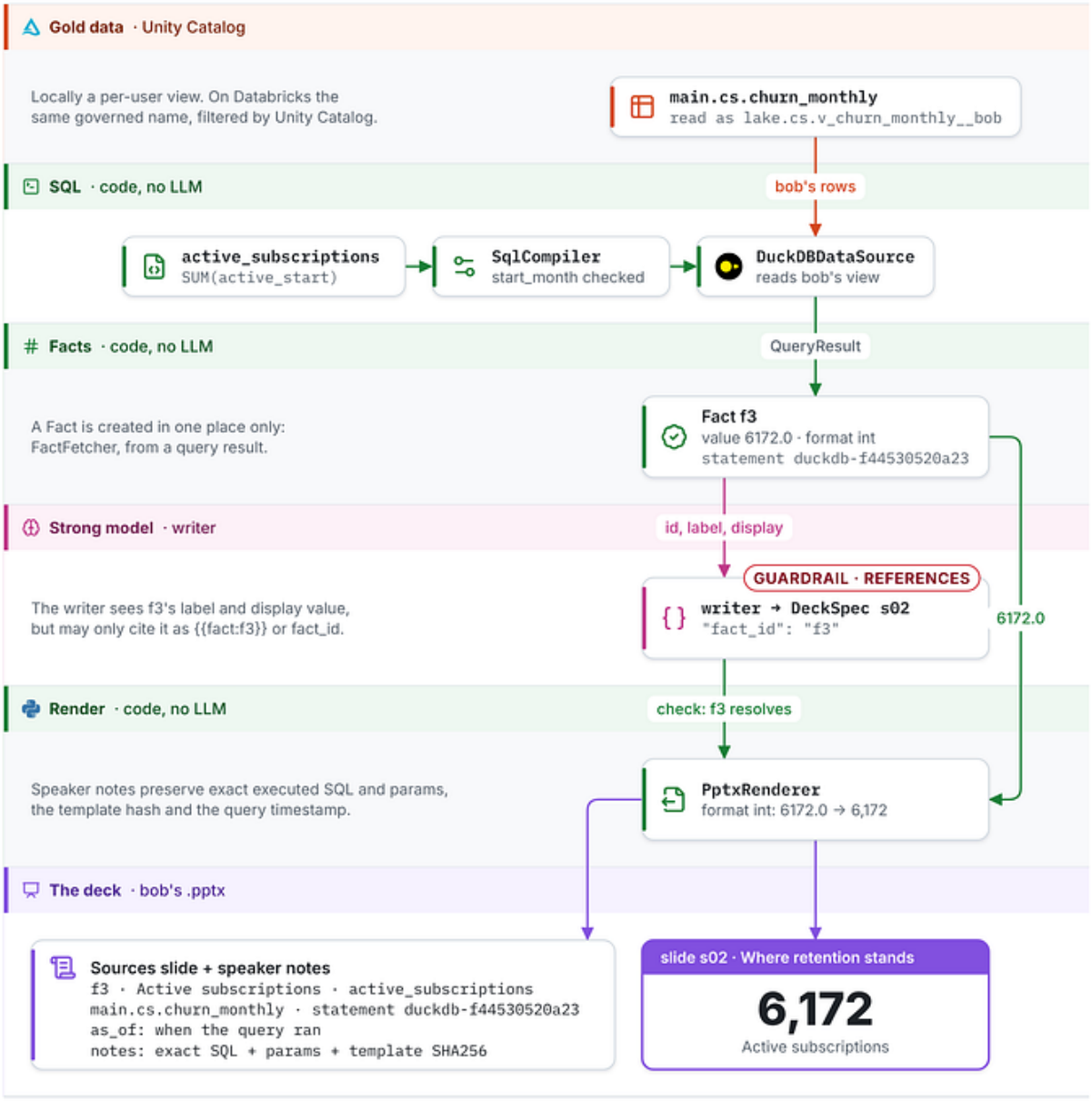
The data-fetch step turns each result into a typed fact. The writer can reference those facts but cannot add one to the registry. Here is a real entry from bob's run, trimmed:

```
{
  "id": "f3",
  "label": "Active subscriptions",
  "value": 6172.0,
  "source": {
    "template_id": "active_subscriptions",
    "tables": ["main.cs.churn_monthly"],
    "as_of": "…"
  }
}
```

The table is recorded by its governed name, even though bob's view was read. The complete source also stores the executed SQL, bound parameters and SQL template hash. `as_of` is the query execution time; it does not establish how recently the gold table was refreshed.

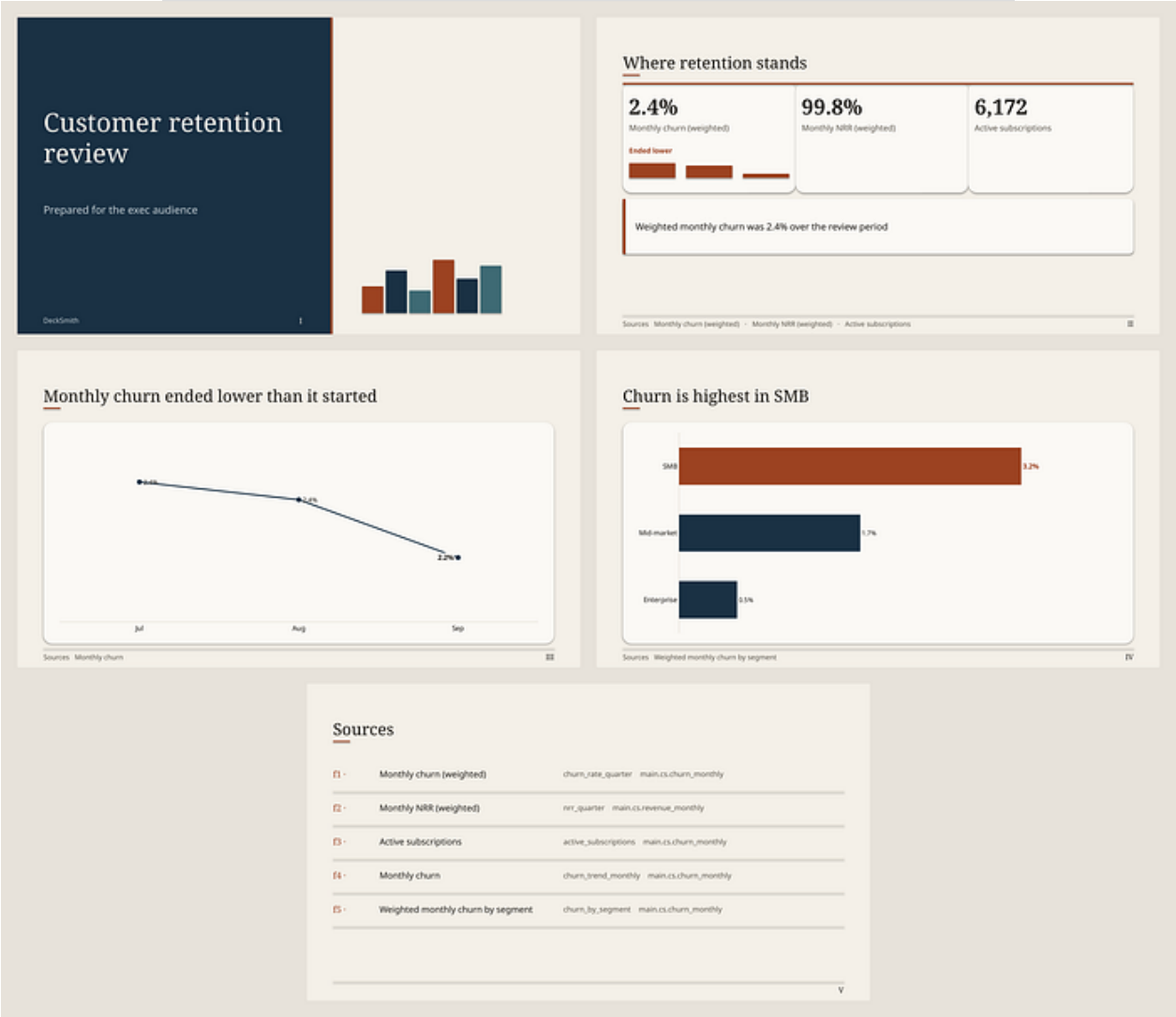
4. The writer references typed facts

The writer doesn't produce slides. It produces a DeckSpec: a strict JSON description of each slide. A KPI holds a fact ID, never a value, and text cites facts by placeholder, as in "Weighted monthly churn was {{fact:f1}} over the review period". The renderer then prints `f1` as 2.4% and `f3` as 6,172.



5. A deterministic renderer

python-pptx (https://python-pptx.readthedocs.io/) turns the spec and facts into a native `.pptx`: charts you can restyle, query details in speaker notes, and a required sources slide. The same spec, facts, theme and run metadata produce the same slide content; tests compare XML content rather than ZIP timestamps.



> For me, the practical benefit of a code renderer is repeatable testing. A fixed rendering path makes changes easy to compare with a golden file. Asking the model to generate rendering code on each run would add another source of variation.

6. One gateway for every model call

With real models enabled, calls go through the Portkey gateway (https://github.com/Portkey-AI/gateway): a small model for planning, a strong one for writing. The example config tries a local Ollama model first and falls back to Claude on Amazon Bedrock once you give the gateway AWS credentials. A second config routes both tiers to OpenAI with your own key. Changing models is a config change.

Defining the review metrics

The quarterly review summarises monthly observations. Its churn KPI is `SUM(churned) / SUM(active_start)`, and its NRR KPI is `SUM(mrr_retained) / SUM(mrr_start)`. Both are weighted monthly ratios over the selected months.

> A traceable query can still answer the wrong business question. In this synthetic dataset, following the July-start cohort to the start of October gives 6.84% churn and 99.43% NRR. Those are different metrics from the monthly ratios shown here. The distinction belongs in the label, not in a footnote.

Guardrails: trust by construction

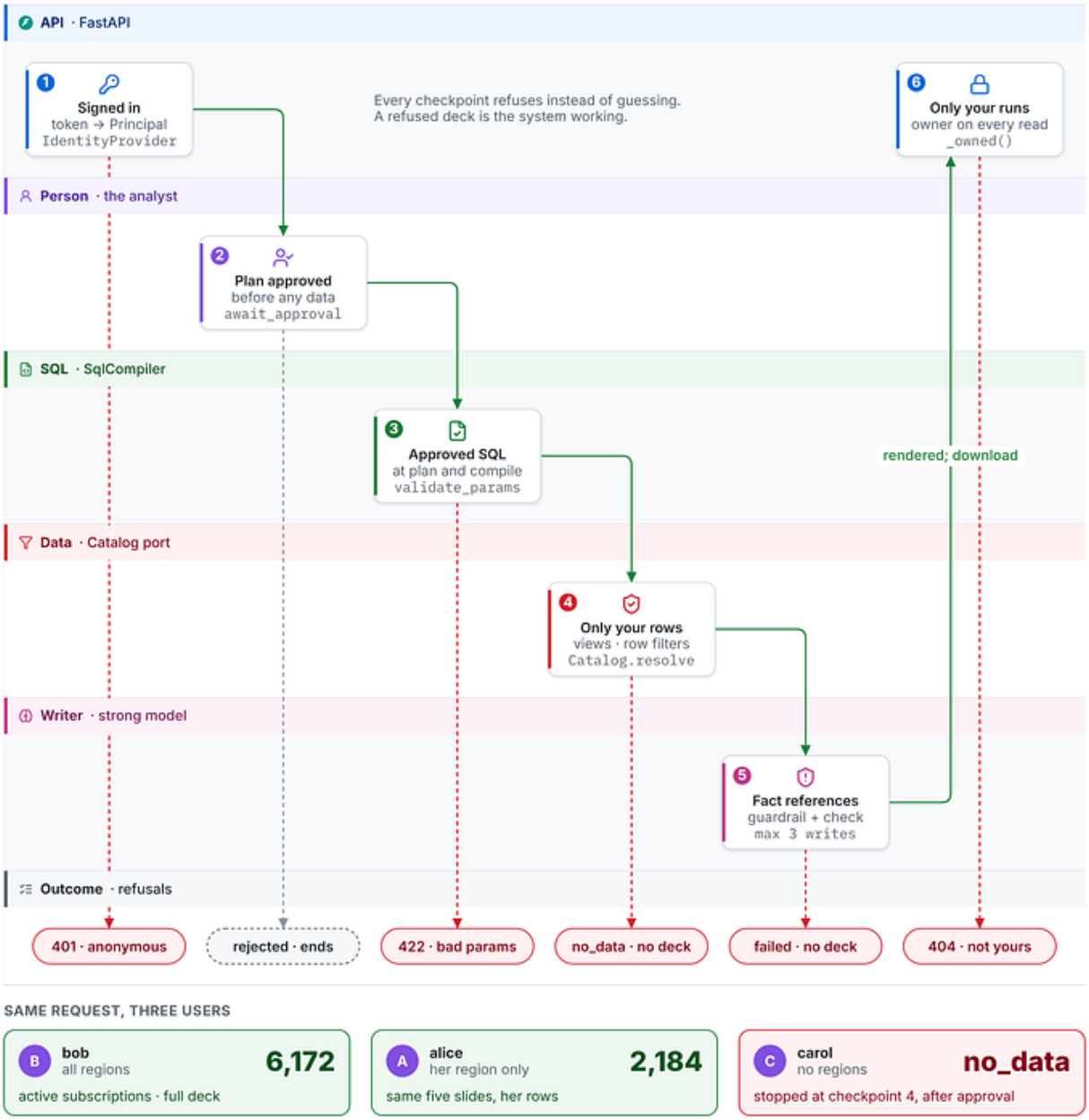
Numeric references are checked twice. First, a guardrail wraps the LLM gateway and rejects digits outside fact placeholders in writer text, including chart series names and notes:

```
# src/decksmith/application/guardrails.py (excerpt)
def complete(self, request: LLMRequest) -> LLMResponse:
    response = self._inner.complete(request)
    if request.node in self._nodes and _has_bare_number(response.text):
        raise GuardrailViolation(f"{request.node}: digits outside {{{{fact:id}}}}
placeholders")
    return response
```

The writer records the rejection and gets another try. Second, deterministic QA confirms that each reference exists, text placeholders use scalar facts, chart series share categories and units, and the sources appendix covers the data. It also checks that slide IDs, order and layouts match the approved outline. After three failed writes, the run stops with no deck. A failed check produces an explicit failure instead of an artifact.

> During review, a chart legend containing an invented percentage passed the original checks. Extending the tests to inspect chart names and plotted values caught a gap that the headline coverage number had missed.

As I see it, these checks enforce a representation rule, not factual correctness of every sentence. Spelled-out quantities, misleading labels and unsupported interpretations still need human review.



Around that core, anonymous calls are rejected and another user's run looks like it doesn't exist. Authentication happens before the ResponsesAgent is traced. The principal travels through request context. A credential in a ResponsesAgent request body is rejected, and credentials stay out of checkpoints and trace inputs. A regression test checks a stored MLflow trace with a dummy token. Provider keys are inserted into gateway configuration in memory. The owner check governs app endpoints; deployment permissions must separately protect checkpoints, traces and saved files.

Same request, three users:

USER	CAN SEE	RESULT
B bob	All regions	A full deck, with 6,172 active subscriptions
A alice	Her region only	A default five-slide deck with her numbers: 2,184
C carol	Nothing	She can approve the outline, then gets a plain refusal and no deck

> Carol's result is my favourite. An empty deck looks like "churn was zero". Locally she has no view, so she reads "None of the requested data is visible to you". On Databricks, the row filter would give her zero rows instead, and she would read "The period may have no data yet, or row-level policies may hide it." Either sentence is the truth.

The refusal also has to stay truthful when nobody is denied. If Bob asks for a quarter with no data yet, the queries run and return nothing, so the message says the period may be empty or hidden by row-level policies instead of blaming his permissions. Any denied or empty query stops the run; DeckSmith never builds a deck from part of the data.

Testing it end to end

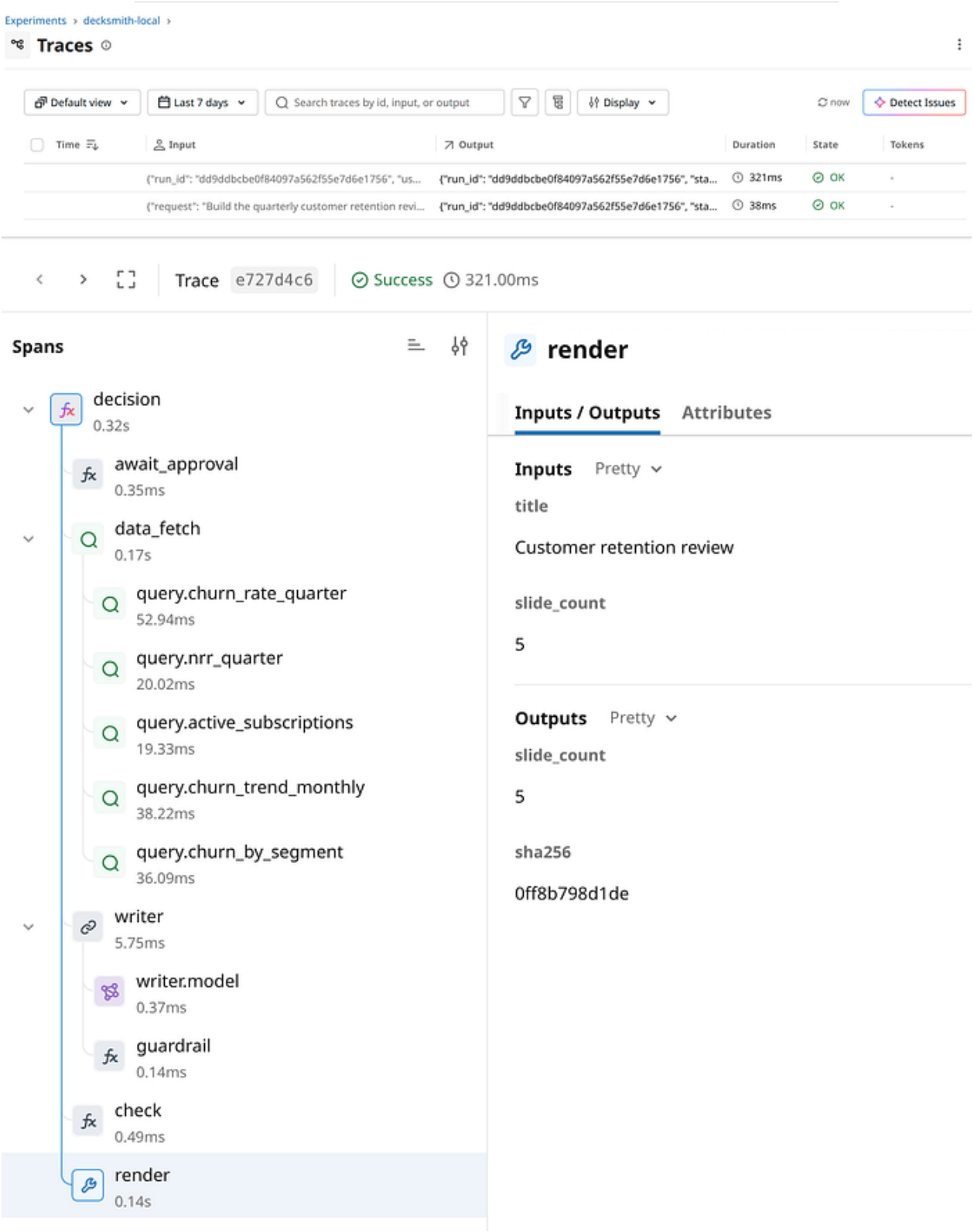
The tests follow the layers of the code, behind a 90% coverage gate. They include regressions for token tracing, numeric references, source coverage, chart alignment, outline edits and refusal messages. The same contract tests run against local and Databricks adapters alike. Ten golden prompts score 100% on deterministic scorers, also through `mlflow.genai.evaluate`. Every run here uses the deterministic stand-in model, so these results test the pipeline and its checks, not a real model.

In CI on the main branch, 259 tests passed and 6 were skipped: five need live services or a workspace, and one needs LibreOffice. Coverage was 97.39%. All 11 steps of the API use-case script passed: it resumes Bob's edited run, verifies its four slides, checks every chart category and plotted value against its fact, then checks Alice's default deck, Carol's refusal and run privacy.

Bob's default deck shows weighted monthly churn of 2.4%, weighted monthly net revenue retention of 99.8%, and 6,172 active subscriptions at the start of July. Monthly churn is 2.4%, 2.4% and 2.2%; weighted monthly churn by segment is 0.5% Enterprise, 1.7% Mid-market and 3.2% SMB. The historical template IDs,

`churn_rate_quarter` and `nrr_quarter`, name the review window; their descriptions and slide labels explicitly identify the monthly aggregation. The sources slide lists the facts; native charts retain their numeric series, and notes include query details.

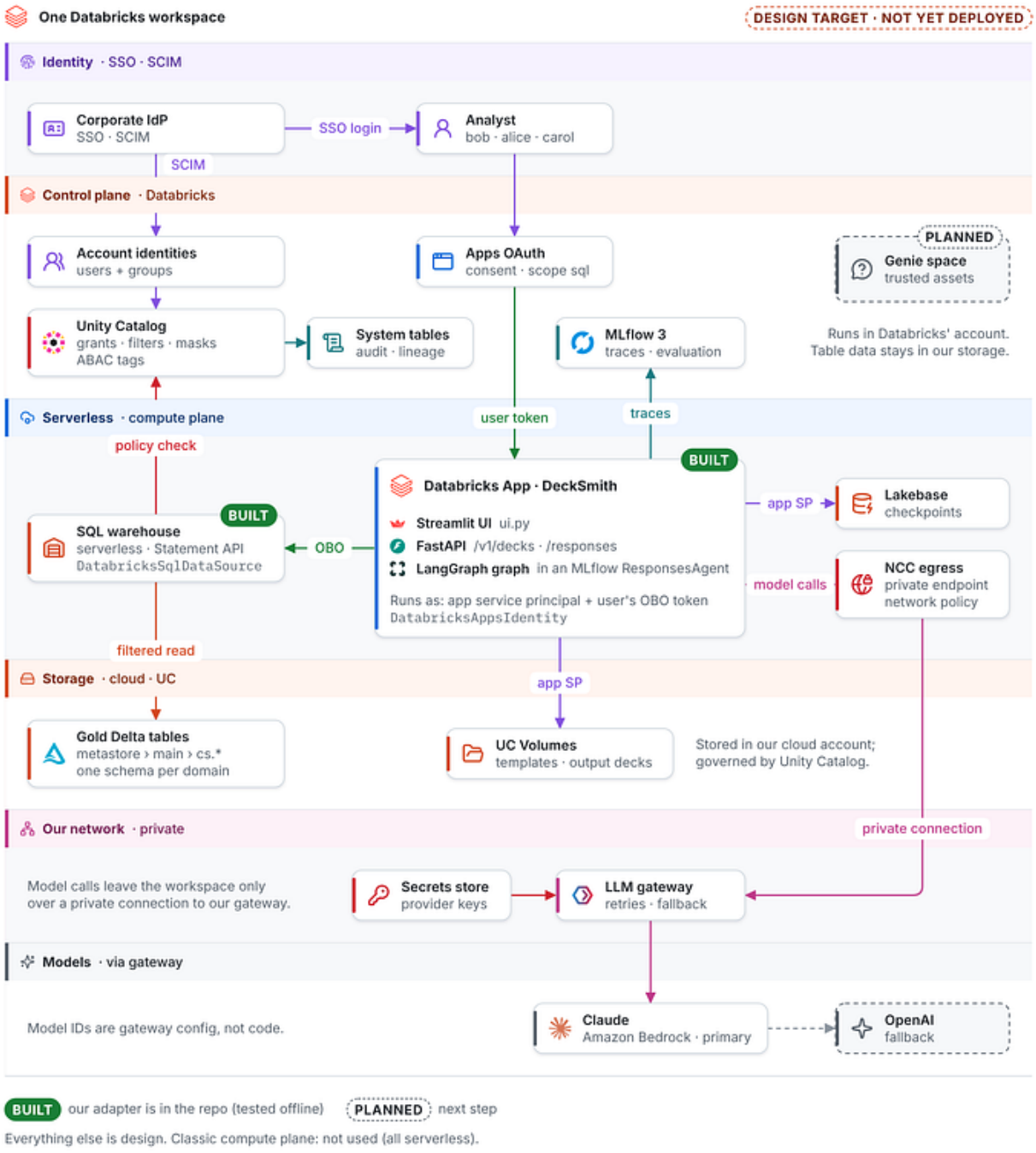
With MLflow tracing on, one deck is two traces, because the approval pause splits the run: `draft` runs until the pause, and `decision` is the resume. Each trace has a root `WORKFLOW` span and a child span per step: `intake`, `planner` and `writer` (`CHAIN`), their model calls (`CHAT_MODEL`), `await_approval` (`TASK`), `data_fetch` and one `query.<template_id>` per template (`RETRIEVER`), `guardrail` and `check` (`GUARDRAIL`), and `render` (`TOOL`). A trace records the request or decision, the acting user, outline and deck summaries, and only the facts that user may see. Tokens, API keys, SQL text, file paths and query timestamps are left out, and long text is truncated.



Local MLflow. Top: one run's draft and decision traces, with request and outcome previews. Bottom: the decision trace's span tree, with the render span selected. The test results above come from CI; the workspace deployment remains untested.

The Databricks deployment design

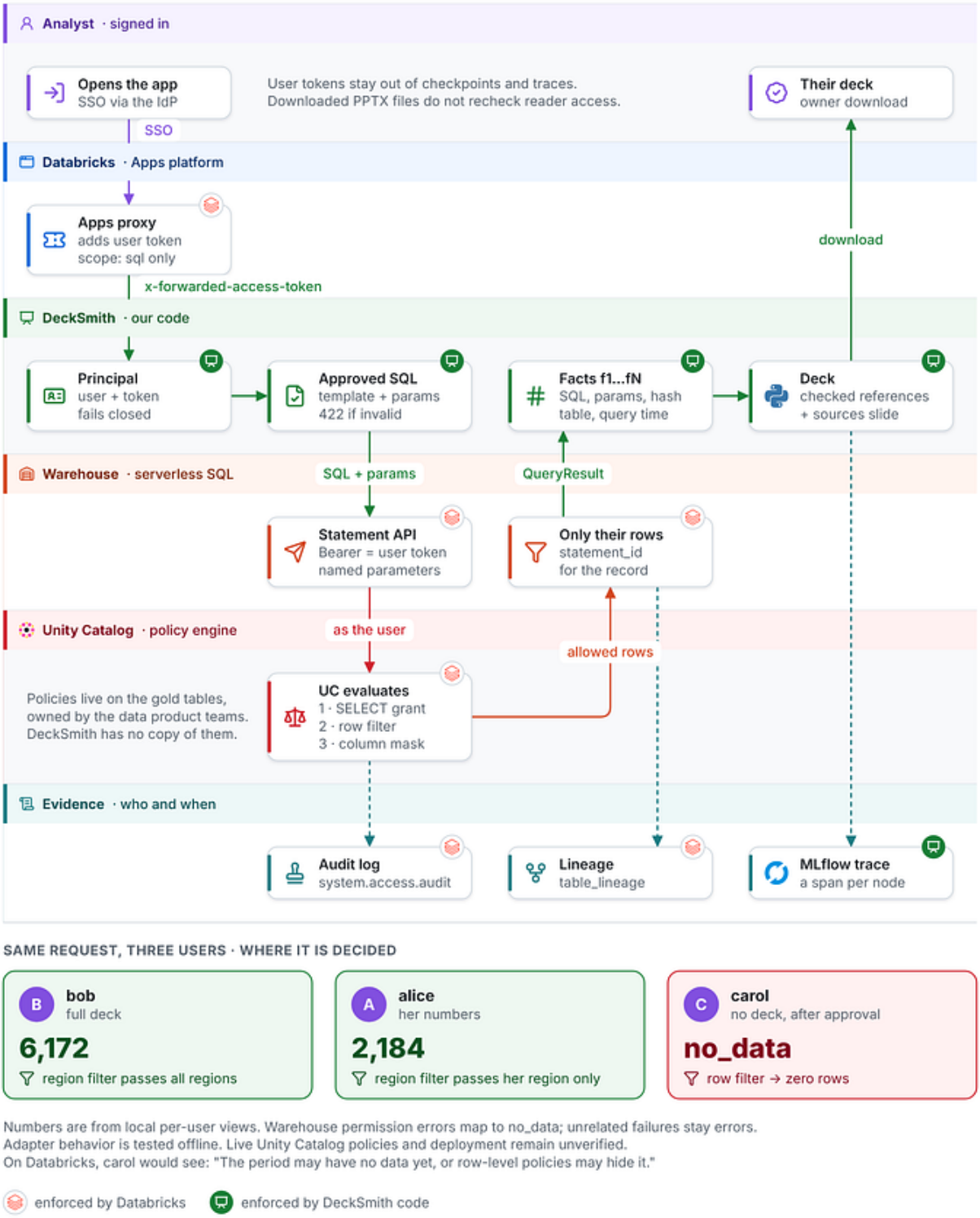
The target is one Databricks workspace, running the same agent with the Databricks adapters switched on.



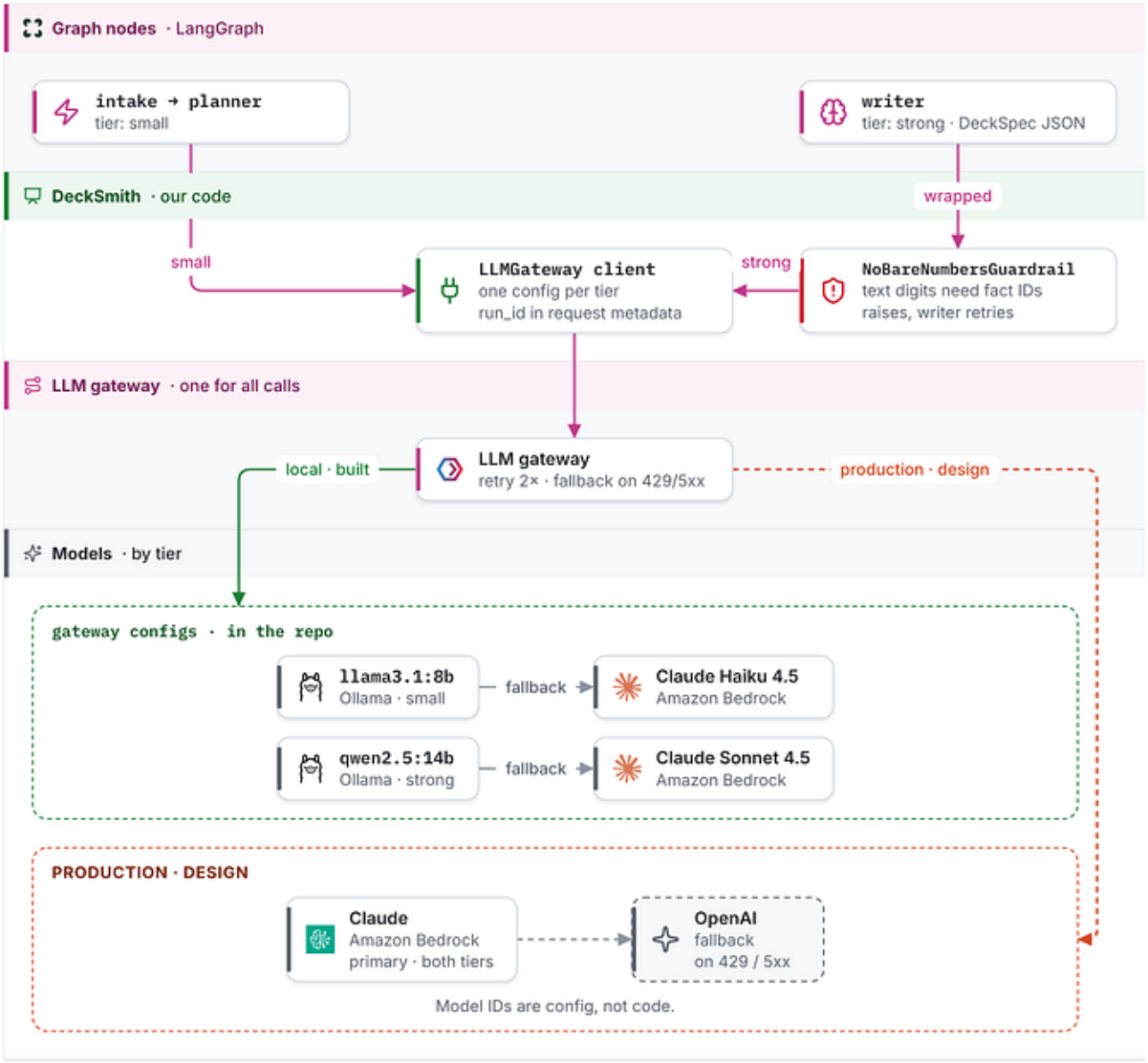
Identity: on behalf of the user. With user authorization enabled, Databricks Apps (https://docs.databricks.com/aws/en/dev-tools/databricks-apps/auth) passes the signed-in user's own short-lived token to the app on each request, limited to the scopes the app declares (Databricks blog (https://www.databricks.com/blog/now-ga-building-permission-aware-databricks-apps-behalf-user-authorization)). DeckSmith's Databricks identity

adapter refuses requests without the forwarded identity headers. Both the UI and API now have a header-forwarding path, tested offline; the trusted Databricks proxy boundary still needs a workspace test.

Data: Unity Catalog decides. Each template runs on a SQL warehouse through the Statement Execution API (https://docs.databricks.com/aws/en/dev-tools/sql-execution-tutorial), with the user's token. Row filters (https://docs.databricks.com/aws/en/data-governance/unity-catalog/filters-and-masks/) (rules that hide rows a user may not see) or tag-based ABAC policies (https://docs.databricks.com/aws/en/data-governance/unity-catalog/abac/) decide which rows come back. Row filters cannot be applied to views, so they go on the gold tables. The data owners keep the policy, and my code stays out of it.



State, traces and models. Saved runs move to Lakebase (https://docs.databricks.com/aws/en/oltp/projects/state-management), which Databricks documents for LangGraph, so a run can wait for approval across restarts. Tracing (https://docs.databricks.com/aws/en/mlflow3/genai/tracing/) and evaluation move to managed MLflow, and decks land in Unity Catalog volumes. Model calls still go through one LLM gateway, with Claude on Amazon Bedrock as primary and OpenAI as the fallback.



Agent hosting

I designed DeckSmith's UI and API to run as a Databricks App, with every request using the signed-in user's own token. The agent is wrapped in MLflow's `ResponsesAgent` and served at `/responses`, which Databricks now documents as its legacy agent-server path (Databricks docs (https://docs.databricks.com/aws/en/agents/custom-agents/author-agent)). For new agents, Databricks recommends the Agent Bricks CLI, which deploys to Agent Runtime, its managed agent hosting (Deploy agents on Databricks (https://docs.databricks.com/aws/en/agents/deploy/)). The recommended server there, `DurableAgentServer`, serves a different API at `/api/invocations`. Acting as the requesting user only works with that recommended server, so DeckSmith, which runs every query as the user, needs an `/api/invocations` adapter on Agent Runtime. That server keeps run state across worker restarts and can recover interrupted runs through a recovery handler, but it does not store the user's credential, so it cannot recover a run that acts as the

requesting user (Agent Server (https://docs.databricks.com/aws/en/agents/deploy/agent-server)). DeckSmith runs every query as the user, so on Agent Runtime an interrupted run would fail and have to be resent. The ports keep hosting concerns outside the graph, but they do not prove compatibility: packaging, the invocation adapter, checkpoint lifecycle and the approval pause still need testing on Agent Runtime.

What still needs a workspace test

The local use case runs end to end and passes. Four things still need proving:

- **The Databricks adapters.** They pass offline tests but have not yet run against a real SQL warehouse.
- **Workspace permissions and persistence.** Mocked tests cover warehouse permission errors, asynchronous queries and result chunks. Actual grants, row filters, token refresh, Lakebase recovery and artifact access still need a workspace run.
- **Real models.** The tests use the deterministic stand-in model, so they test local permission emulation and numeric-reference rules, not real-model writing quality. The Bedrock fallback is configured but untested, and it needs AWS credentials that the repo does not supply.
- **Hosting.** Forwarded-header authentication is implemented for the UI and `/responses`, but has not been exercised behind the Databricks Apps proxy. On Agent Runtime, acting as the user requires the recommended server, so DeckSmith needs an `/api/invocations` adapter that does not exist yet, and the approval pause and the loss of crash recovery for user-authorized runs have not been tried there.

Lessons for a team starting this

If your business asks for "AI-generated slides", treat it as a governed data product with a renderer, not as a prompt:

1. **Start from gold data products.** The deck is a view of gold, not a second truth.
2. **Treat numeric metrics as typed data.** Fact IDs let tests check values, references and chart categories. A reviewer still needs to assess the story.
3. **Approve before you fetch.** Agreeing the storyline first saves queries and earns trust.
4. **Run every query as the user, and refuse plainly.** No data means no deck, and a sentence that says why.
5. **Start local, but behind ports, and label your emulations.** Local first only pays off if local is shaped like production, and you say where it isn't.

What's next

- **A real workspace and real models:** close the gaps above, on Amazon Bedrock and synthetic data, and add LLM-judge scorers.
- **Genie trusted assets**, verified example queries, as a second governed source of facts.
- **The knowledge base** from the previous article (https://freedium-mirror.cfd/https://medium.com/data-engineer-things/build-a-document-knowledge-base-on-databricks-2e2914fb8d4e), as context for the writer, never as a source of numbers.

Try it yourself

You need Python 3.11+, `make` and git. Docker is optional.

```
git clone https://github.com/santoshshinde2012/deck-smith-local.git && cd deck-smith-local
make quickstart        # install → seed → start API and UI → build decks; opens http://localhost:8501
make e2e-usecase       # asserts the whole use case and prints a pass/fail table
make stop              # stop everything
```

You should see the three results above (6,172 for bob, 2,184 for alice, a refusal for carol) and 11/11 steps passed. The quickstart uses the same stand-in model, so your numbers match these exactly. The repo README shows how to switch on a real model through the gateway.

Final thought

For me, the useful boundary is concrete: approved queries supply the facts, the requesting user determines which data can enter a run, and code checks the references and renders the file. That makes my local prototype auditable. Business trust still depends on clear metric definitions, reviewed commentary and a separately validated deployment.

The code is on GitHub: deck-smith-local (https://github.com/santoshshinde2012/deck-smith-local). Clone it, run `make quickstart`, and tell me what your business would ask for first. If it helps you, a star on the repo helps others find it.

I also share a free learning path for developers moving into AI/ML: ai-ml-ground-truth (https://github.com/santoshshinde2012/ai-ml-ground-truth). My open-source work is on GitHub (https://github.com/santoshshinde2012), I write on Medium (https://blog.santoshshinde.com/), and you can say hello on LinkedIn (https://www.linkedin.com/in/shindesantosh/).